# Aula 5 – Árvores de Pesquisa Binária

Inserção

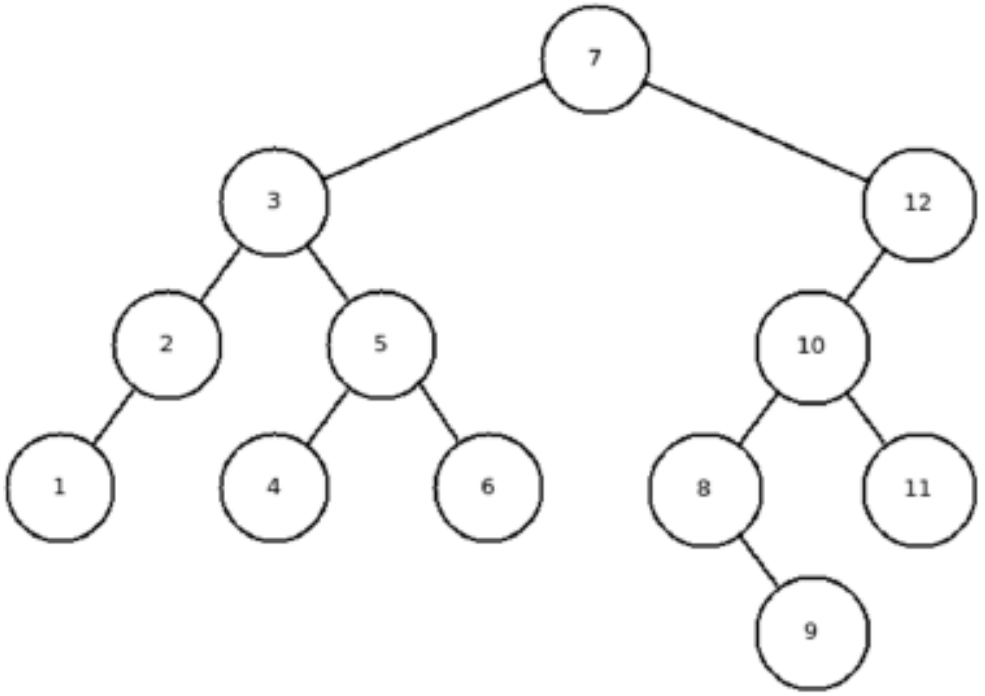
Pesquisa

Eliminação

Exemplos

# Árvores de Pesquisa Binária - APB

## Conceitos




Todos os nodos à esquerda são menores.

Todos os nodos à direita são maiores

Raiz: Nodo inicial (7)

Nodos: Todos os elementos da árvore

Folhas: Nodos terminais (1, 4, 6, 9, 11)

Descendentes: Todos os nodos abaixo (o 2 e 5 são descendentes do 3)

Ascendentes: Todos os nodos acima (o 7 é ascendente do 3 e do 12)

Irmãos: Nodos com o mesmo ascendente (o 3 e 12, o 2 e 5, o 8 e 11)

Nodos Interiores: Todos os Nodos que não são Folhas

Caminho: Nodos Percorridos até chegar ao Nodo pretendido (para ao nodo 11 o caminho é 7, 12, 10, 11)

Ramo: Sub-árvore ( os nodos 4, 5, e 6 são um ramo da árvore, tal como os nodos 8, 9, 10, 11, 12)

Grau: Número máximo de descendentes (sempre 2 nas Árvores de Pesquisa Binária)

Nível: Nível Hierarquico (Nivel 0: 7; Nivel 1: 3, 12; Nivel 2: 2, 5, 10; Nivel 3: 1, 4, 6, 8, 11; Nivel 4: 9)

Altura: Maior Nivel (4)

# Árvores de Pesquisa Binária - APB

Inserção O(logn) - Remoção O(logn) - Pesquisa O(logn)

Métodos:

- void add(Object o); // adiciona um objecto na árvore
- boolean contains(Object o); // se contem um objecto
- boolean remove(Object o); // remove um objecto
- Object findMin(); // Devolve o menor objecto da árvore
- Object findMax(); // Devolve o maior objecto da árvore

# APB - Representação

Nodo {

Object data

Nodo left

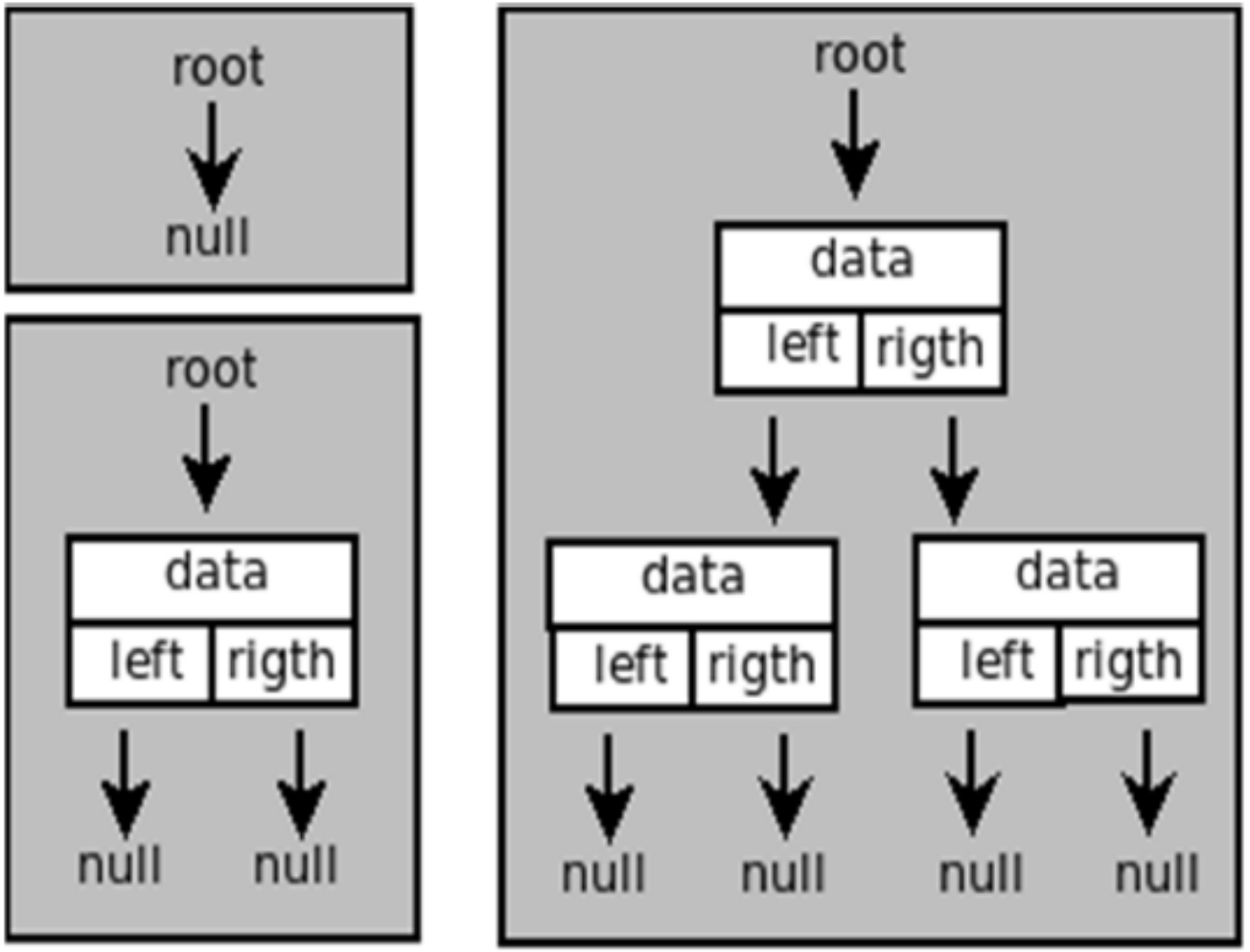
Nodo rigth

}

Tree {

Nodo root

}



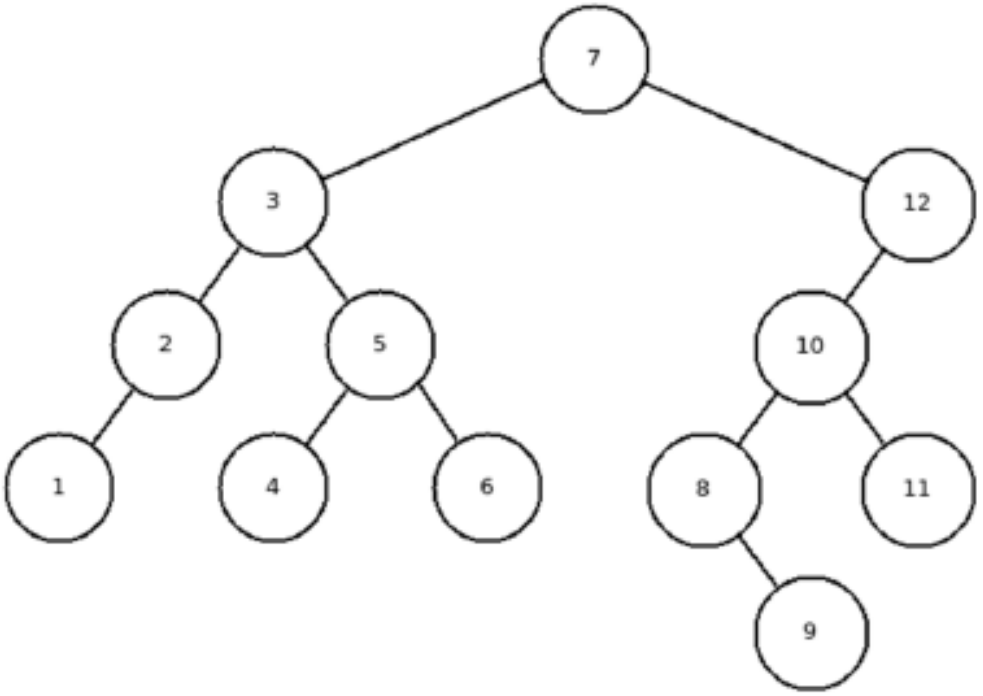
# APB - Inserção

```
void add(Object o) {
    novo(nodo)
    nodo.data = o
    nodo.left = null
    nodo.rigth = null
    Se root = null
        root = nodo
    Senão
        add(root, nodo)
}
```

```
void add(Nodo actual, Nodo novo) {
    Se novo.data < actual.data
        Se actual.left = null
            actual.left = novo
        Senão
            add(actual.left, novo)
    Senão Se actual.rigth = null
        actual.rigth = novo
    Senão
        add(actual.rigth, novo)
}
```

# APB - Pesquisa

```
boolean contains(Object o) {
devolve contains(root, o)
}
```




```
boolean contains(Node actual, Object
    o) {
    Se actual = null
        devolve falso
    Senão Se actual.data = o
        devolve verdadeiro
    Senão Se o.data < actual.data
        devolve contains(actual.left, o)
    Senão
        devolve contains(actual.rigth, o)
}
```

# APB - Remoção

boolean remove(Object o) {

Devolve remove(root, o)

}

boolean remove(Node actual, Object o) {

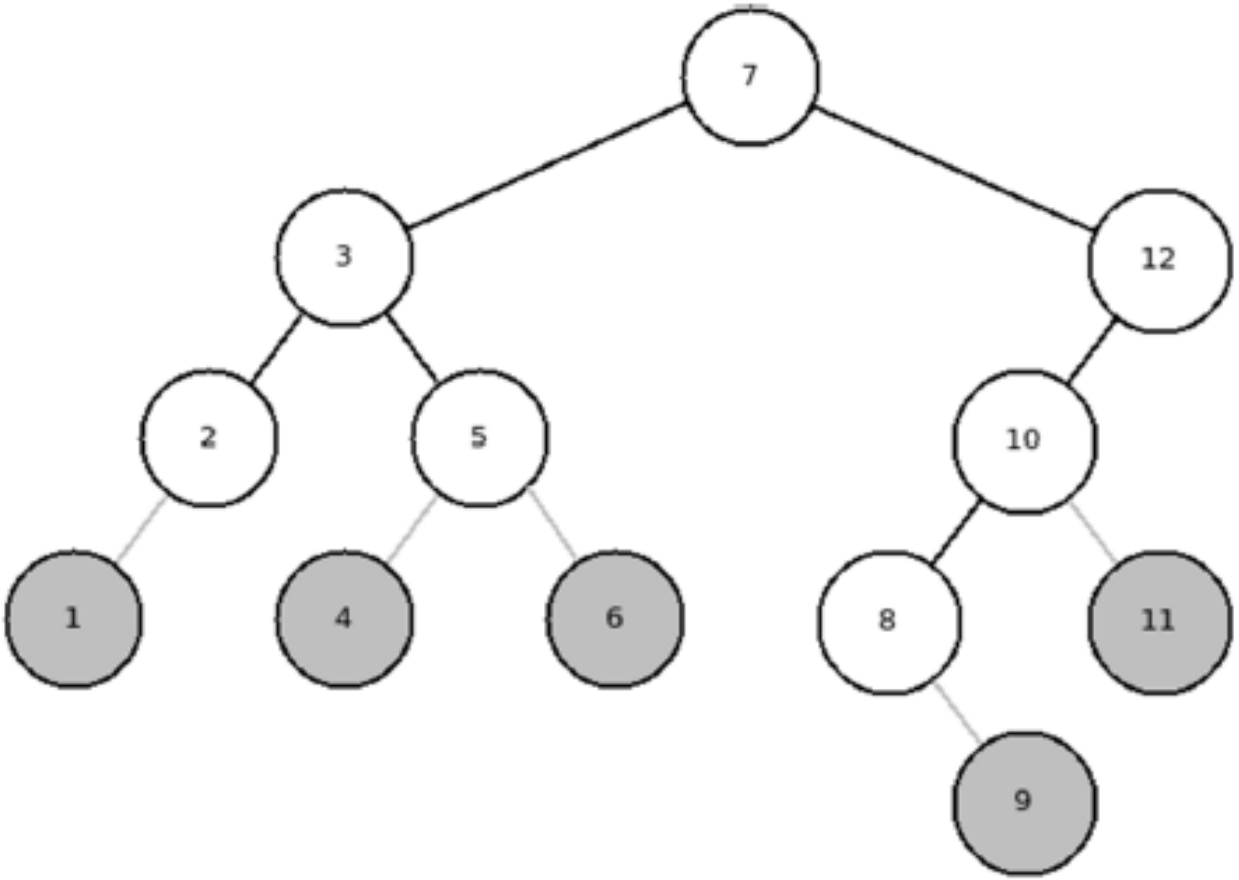
É necessário verificar as seguintes situações:

1. Se o nodo a eliminar é uma folha
2. Se o nodo a eliminar não é uma folha e tem apenas um descedente
   2.1. Se tem um descedente à direita
   2.2. Se tem um descedente à esquerda
3. Se o nodo a eliminar não é uma folha e tem dois descendentes

}

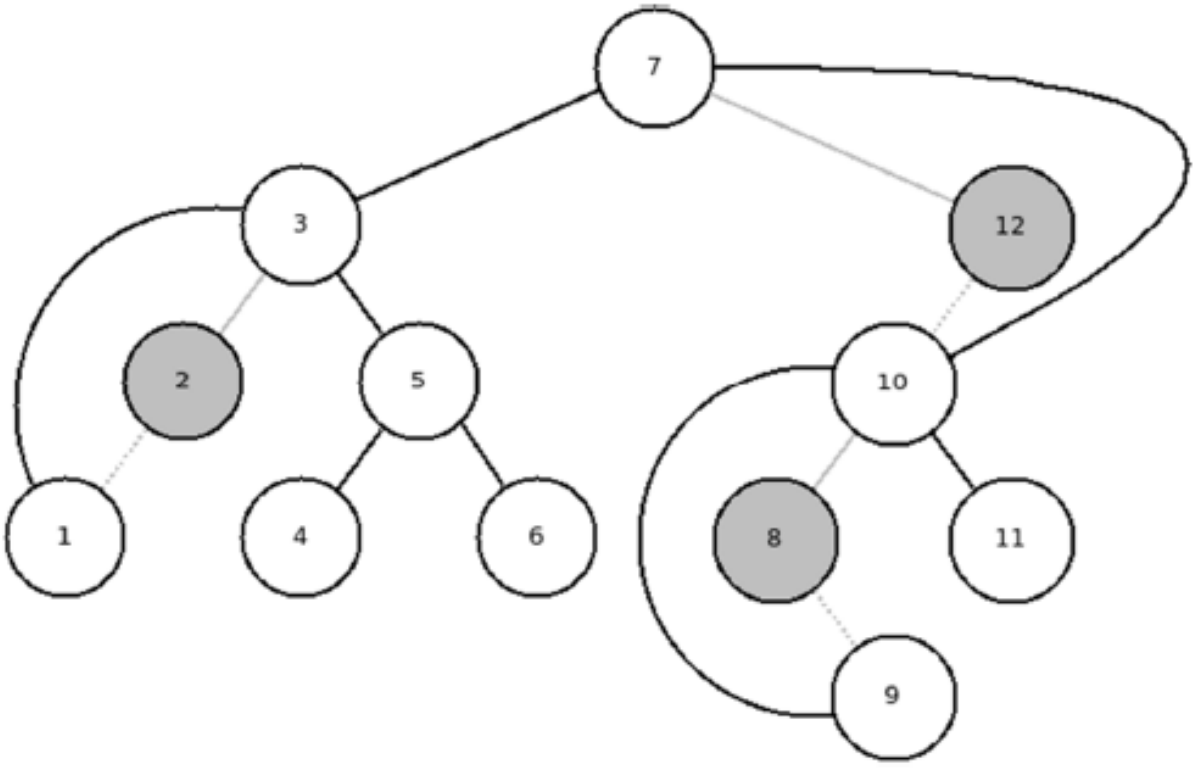
# APB - Remoção

1. Se o nodo a eliminar é uma folha



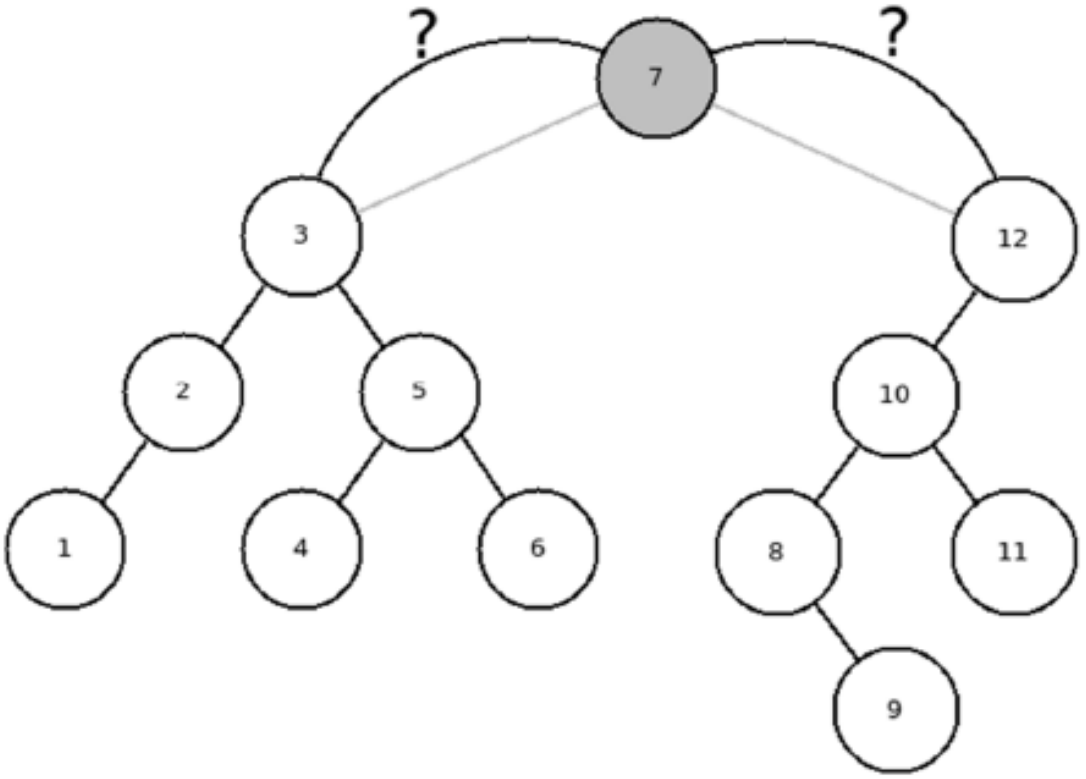
# APB - Remoção

2. Se o nodo a eliminar não é uma folha e tem apenas um descente



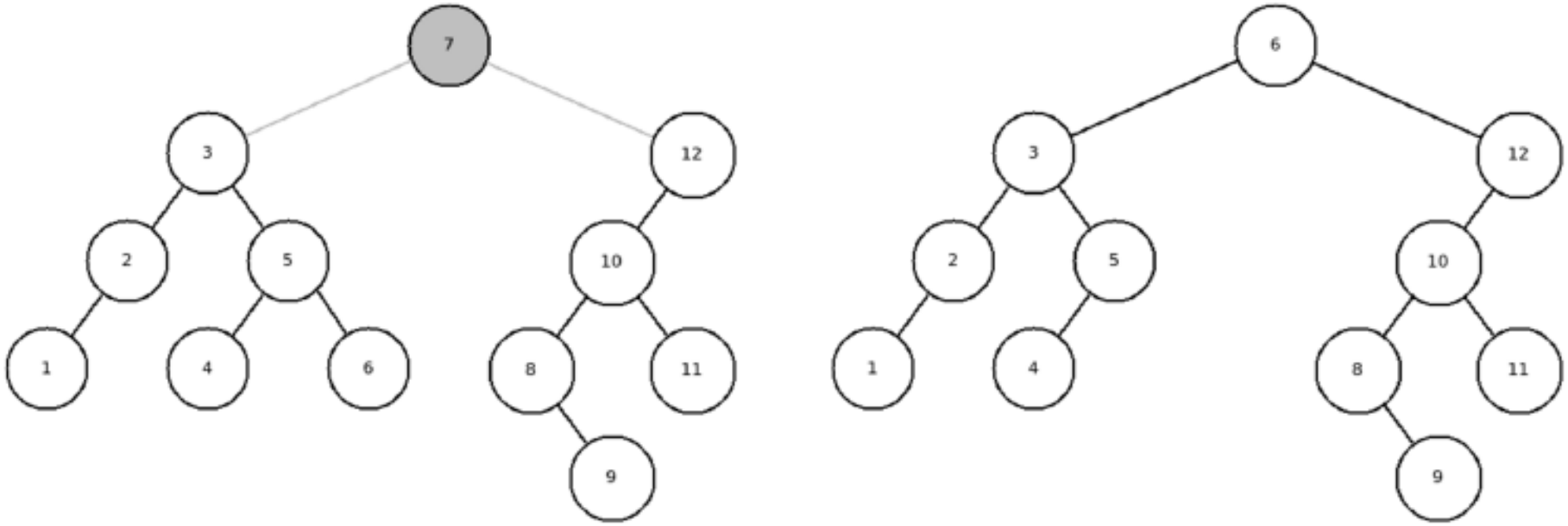
# APB - Remoção

3. Se o nodo a eliminar não é uma folha e tem dois descendentes



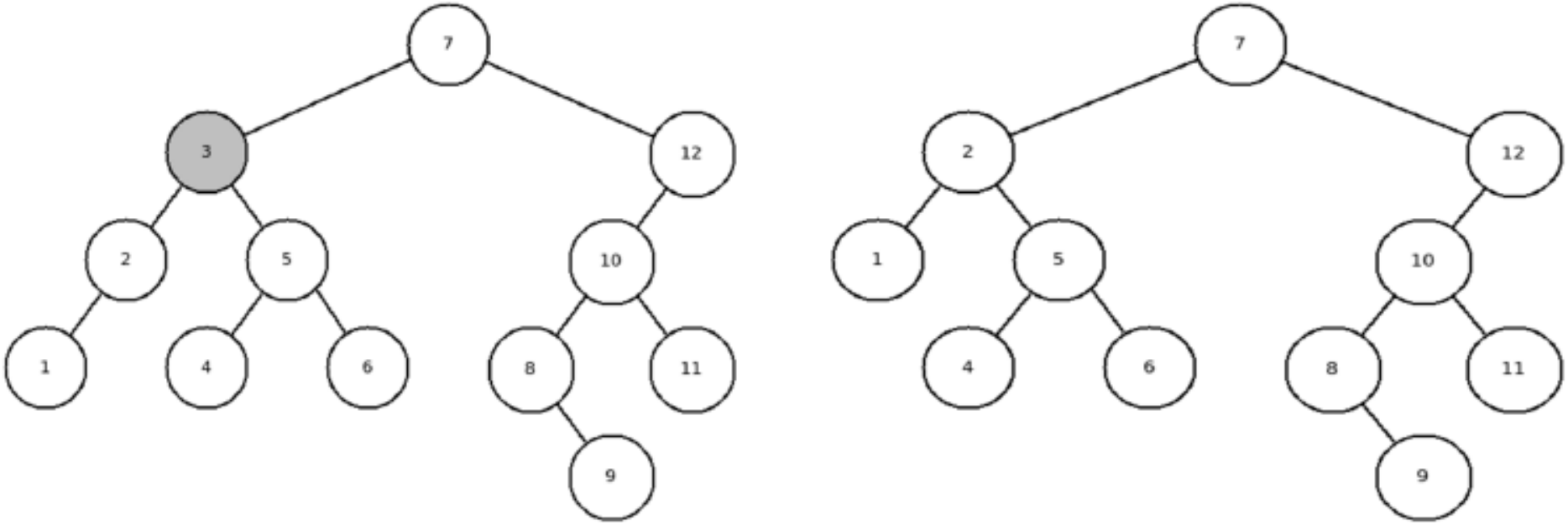
# APB - Remoção

3. Se o nodo a eliminar não é uma folha e tem dois descendentes. Promove-se o maior elemento na sub-árvore esquerda



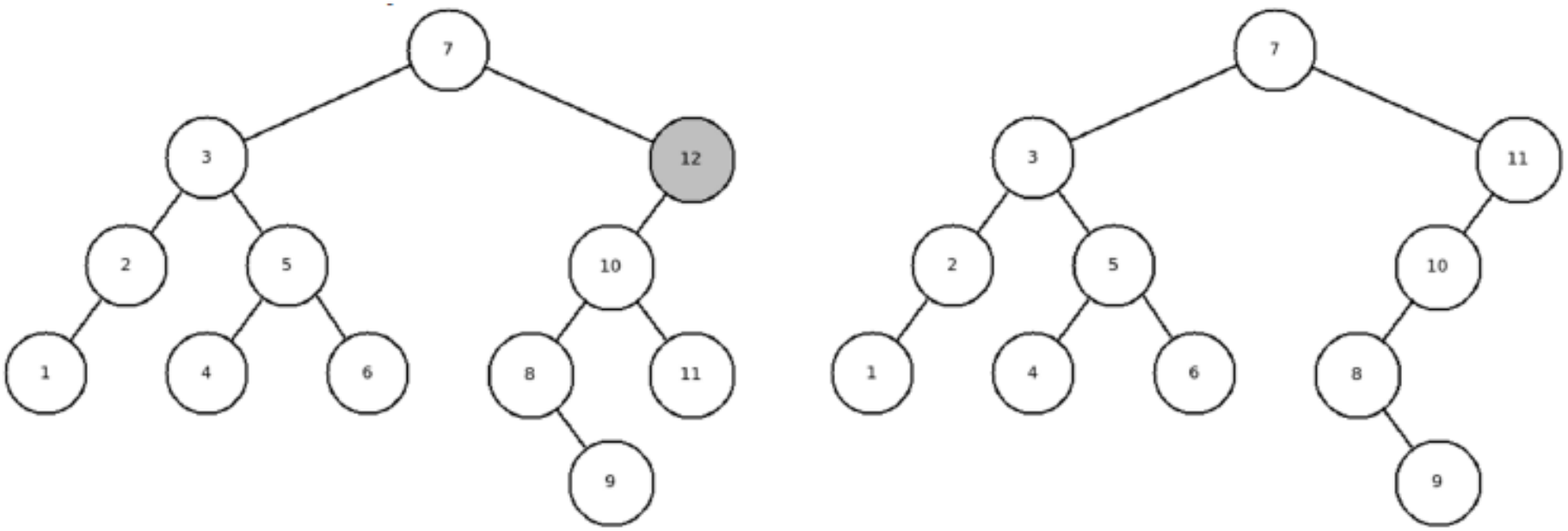
# APB - Remoção

3. Se o nodo a eliminar não é uma folha e tem dois descendentes. Promove-se o maior elemento na sub-árvore esquerda



# APB - Remoção

3. Se o nodo a eliminar não é uma folha e tem dois descendentes. Promove-se o maior elemento na sub-árvore esquerda



# APB - Remoção

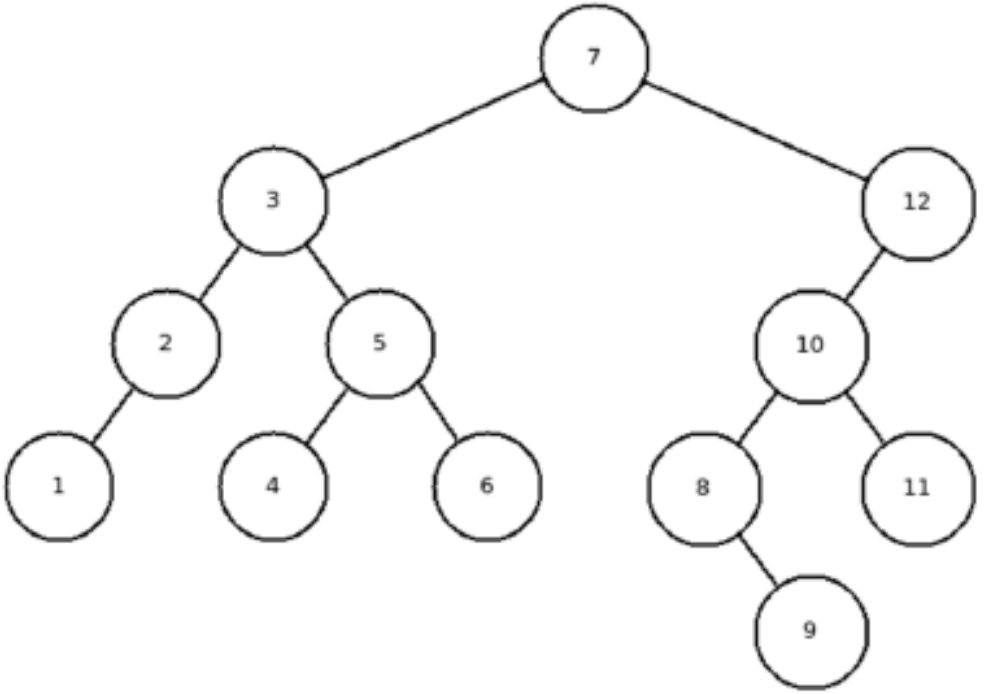
Antes de analisarmos o algortimo para remoção, torna-se necessário compreender dois algortimos:

- Nodo findParent(Object o);

  Devolve o Nodo Ascendente do Nodo que possui o objecto passado por parametro
- Nodo find(Object o);
- Devolve o Nodo que possui o objecto passado por
- parametro

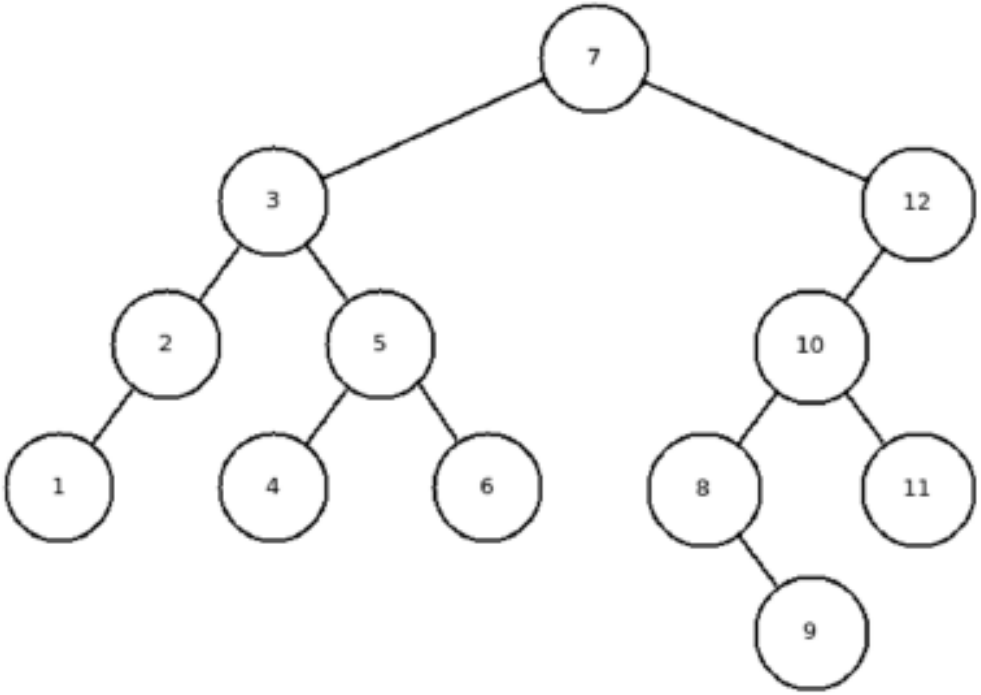
# APB - FindParent

```
Nodo findParent(Object o) {
    Devolve findParent(root, o)
}
Nodo findParent(Nodo actual, Object o) {
    Se o = actual.data
        Devolve null
    Senão
        Se o < actual.data
            Se actual.left = null
                Devolve null
            Senão Se actual.left.data = o
                Devolve actual
            Senão
                Devolve findParent(actual.left, o)
        Senão Se actual.rigth = null
             Devolve null
         Senão Se actual.rigth.data = o
             Devolve actual
         Senão
            Devolve findParent(actual.rigth, o)
}
```



# APB - Find

```
Nodo find(Object o) {
    Devolve find(root, o)
}
Nodo find(Nodo actual, Object o) {
    Se actual = null
        Devolve null
    Senão
        Se actual.data = o
            Devolve actual
        Senão
            Se o < actual.data
                Devolve find(actual.left, o)
            Senão
                Devolve find(actual.rigth, o)
}
```



# APB - Remoção

```
boolean remove(Node actual, Object o) {
    nodoARemover = find(o)
    Se nodoARemover = null
        Devolve falso
    Senão
        parent = findParent(o);
        Se root.left = null e root.rigth = null
            root = null
        Senão Se nodoARemover.left = null e nodoARemover.rigth = null // caso 1
            Se nodoARemove.data < parent.data
                parent.left = null
            Senão
                parent.rigth = null
```

# APB - Remoção

```
    Senão Se nodoARemover.left = null e nodoARemover.rigth <> null // caso 2.1
       Se nodoARemover.data < parent.data
          parent.left = nodoARemover.rigth
       Senão
          parent.rigth = nodoARemover.rigth
    Senão Se nodoARemover.left <> null e nodoARemover.rigth = null // caso 2.2
       Se nodoARemover.data < parent.data
          parent.left = nodoARemover.left
       Senão
          parent.rigth = nodoARemover.left
     Senão // caso 3
          nodoMaior = nodoARemover.left
          Enquanto nodoMaior.rigth <> null
             nodoMaior = nodoMaior.rigth
          parentNodoMaior = findParent(nodoMaior.value)
          Se parentNodoMaior = nodoARemover
               parentNodoMaiort.left = nodoMaior.left
          senão
                    parentNodoMaior.rigth = nodoMaior.left
          nodoARemover.data = nodoMaior.data
   Devolve verdadeiro
}
```

# APB – findMin & findMax

```
Object findMin() {
    Se root = null
       Devolve null
    Senão
        Devolve findMin(root)
}

Object findMin(Node actual) {
    Se actual.left = null
        Devolve actual.data
    Senão
        Devolve findMin(actual.left)
}
```

```
Object findMax() {
    Se root = null
       Devolve null
    Senão
        Devolve findMax(root)
}

Object findMax(Node actual) {
    Se actual.rigth = null
        Devolve actual.data
    Senão
        Devolve findMax(actual.rigth)
    }
```

# APB – Percorrer uma árvore

Existe três formas de percorrer uma árvore de pesquisa binária:

- PreOrder (RED): Primeiro trata o valor do nodo, depois visita a sub-arvore esquerda e depois a sub-arvore direita
- InOrder (ERD): Primeiro visita a sub-arvore esquerda, depois trata o valor do nodo e finalmente visita a sub-arvore direita
- PostOrder (EDR): Primeiro visita a sub-arvore esquerda, depois a sub-arvore direita e finalmente trata o valor do nodo.

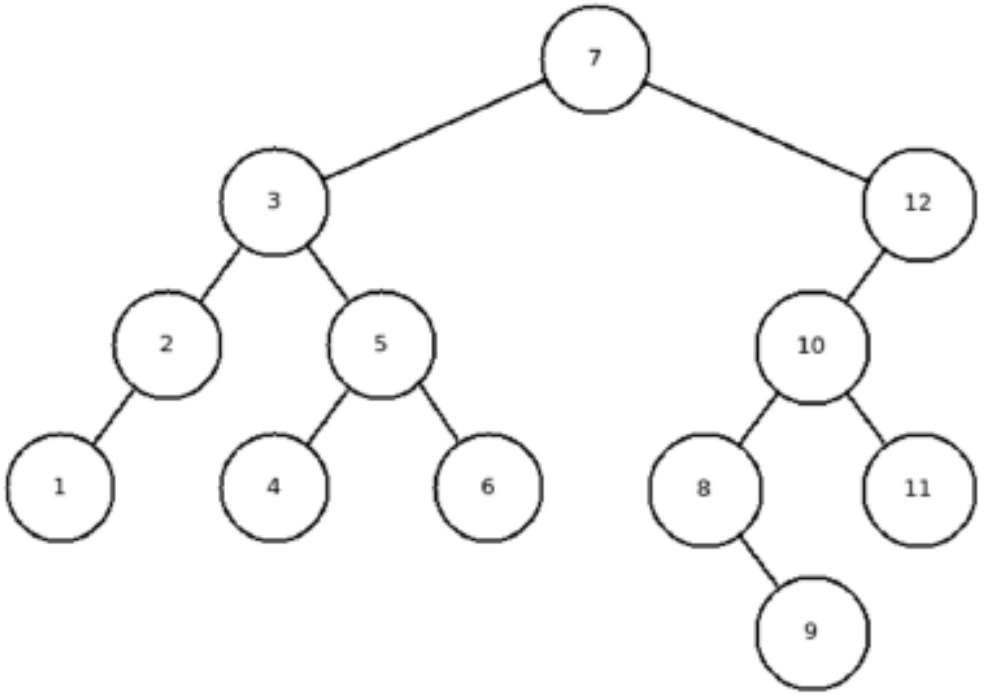
# APB – Percorrer uma árvore

Exemplos de uma travessia

PreOrder (RED): 7, 3, 2, 1, 5, 4, 6, 12, 10, 8, 9, 11

InOrder (ERD): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

PostOrder (EDR): 1, 2, 4, 6, 5, 3, 9, 8, 11, 10, 12, 7



# APB – PreOrder

```
preOrder() {
  preOrder(root)
}

preOrder(Node actual) {
  Se actual <> null
      //trata actual.data
      preOrder(actual.left)
      preOrder(actual.rigth)
}
```

# APB – InOrder

```
inOrder() {
  inOrder(root)
}

inOrder(Node actual) {
  Se actual <> null
      inOrder(actual.left)
      //trata actual.data
      inOrder(actual.rigth)
}
```

# APB – PostOrder

```
postOrder() {
  postOrder(root)
}

postOrder(Node actual) {
  Se actual <> null
      inOrder(actual.left)
      inOrder(actual.rigth)
      //trata actual.data
}
```

# Exemplos – Huffman

Quanto espaço ocupa a frase:

o popo do papa leva o bebe

26 caracteres (o espaço tambem conta) x 8 bits = 208 bits

Número de ocorrencias:

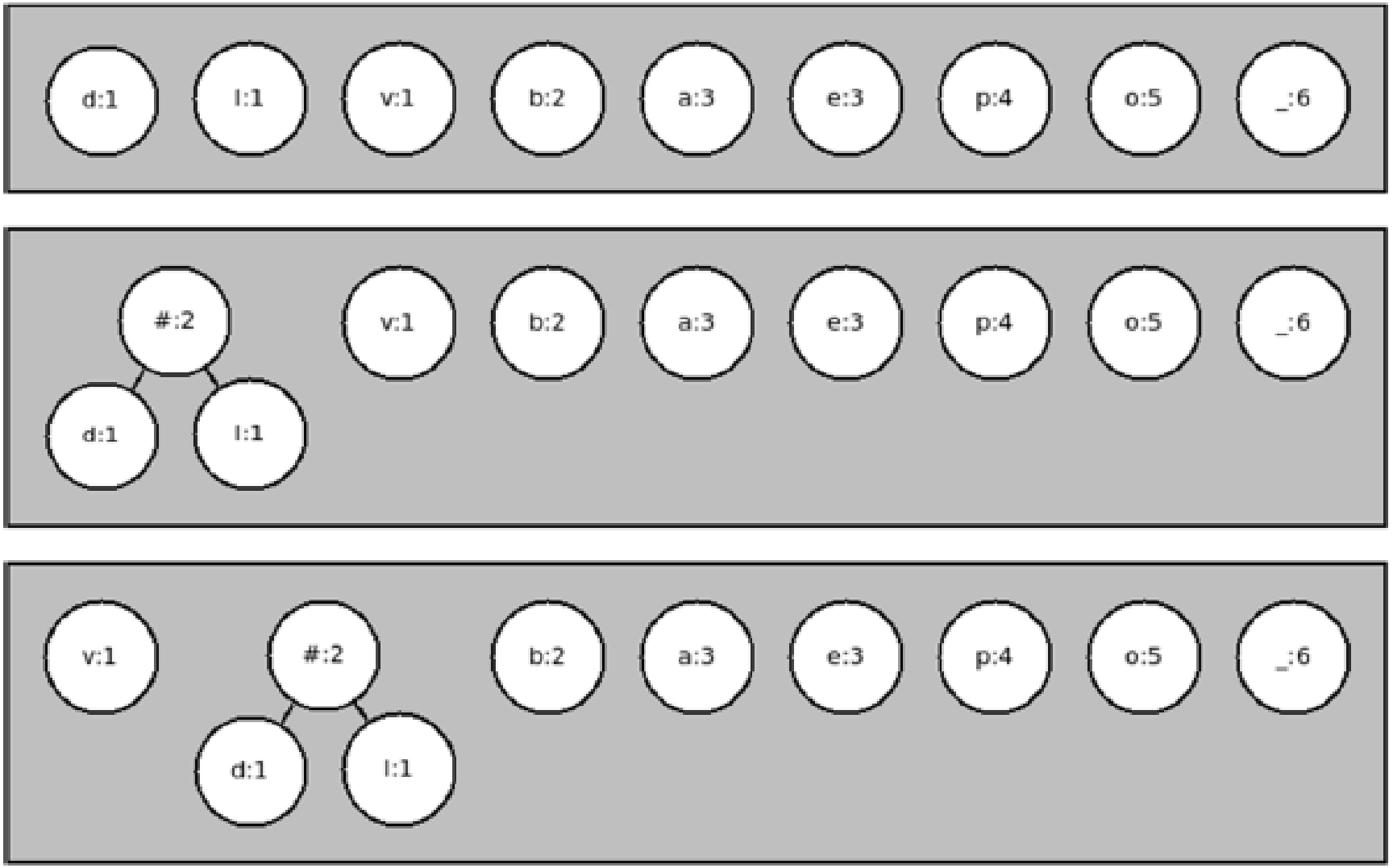
```
o - 5 ocorrências
  - 6 ocorrências
_
p – 4 ocorrências
d – 1 ocorrência
a – 3 ocorrências
l – 1 ocorrência
e – 3 ocorrências
v – 1 corrência
b – 2 ocorrências
```

1º passo: ordenar por ordem crescente todas as ocorrencias. Em caso de empate ordenar por order alfabética

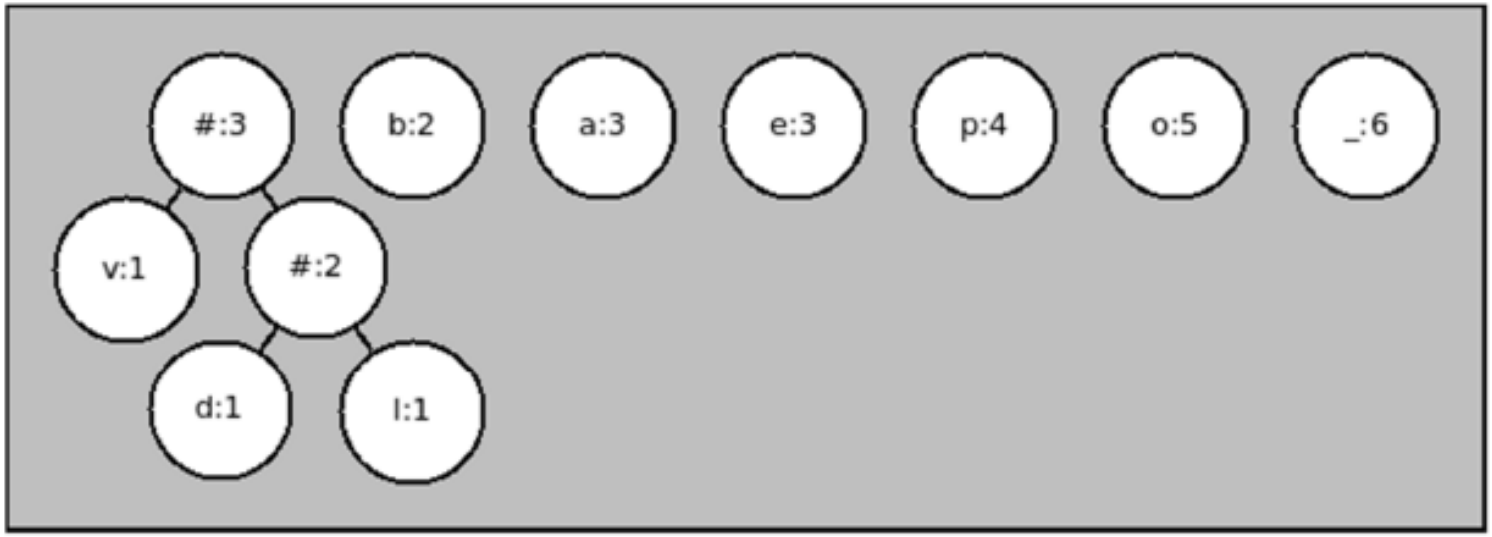
2º passo: criar uma árvore com os dois primeiro elementos, em que a raiz tem a soma das duas ocorrencias

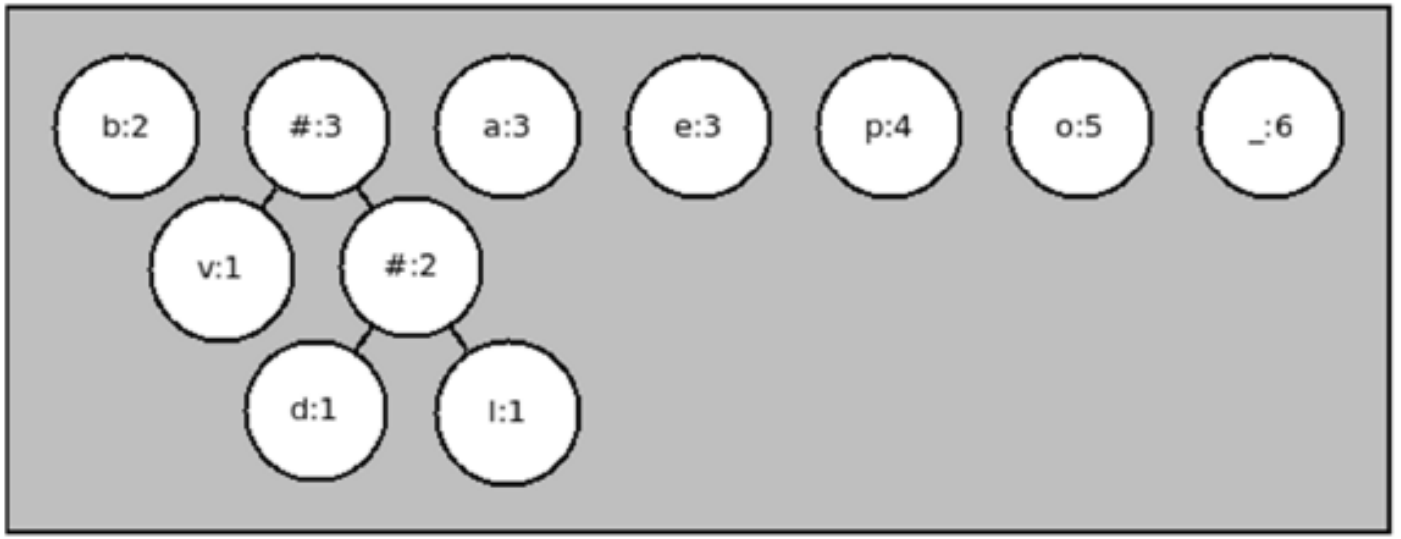
3º passo: voltar a ordenar por ordem crescente

# Exemplos – Huffman

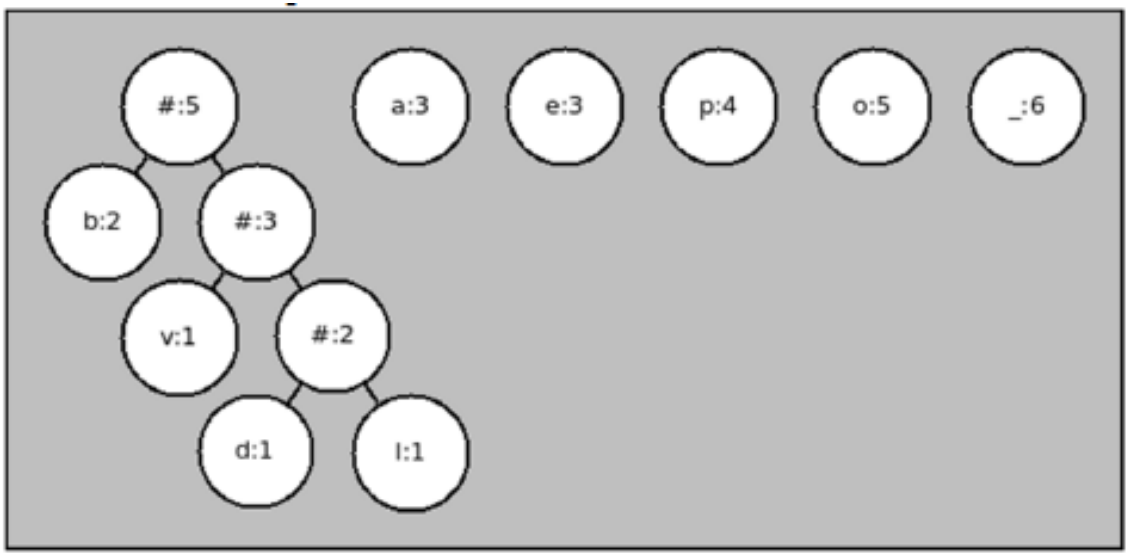


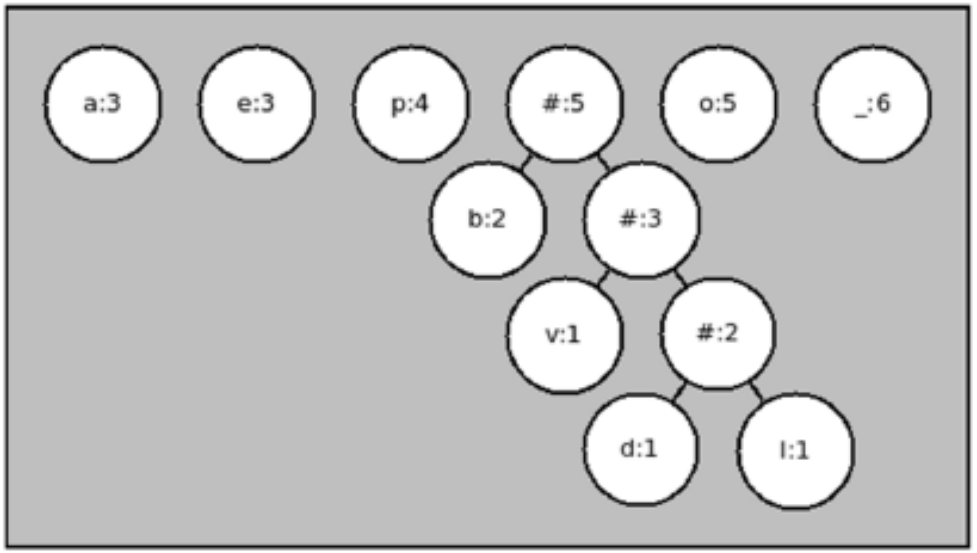
# Exemplos – Huffman



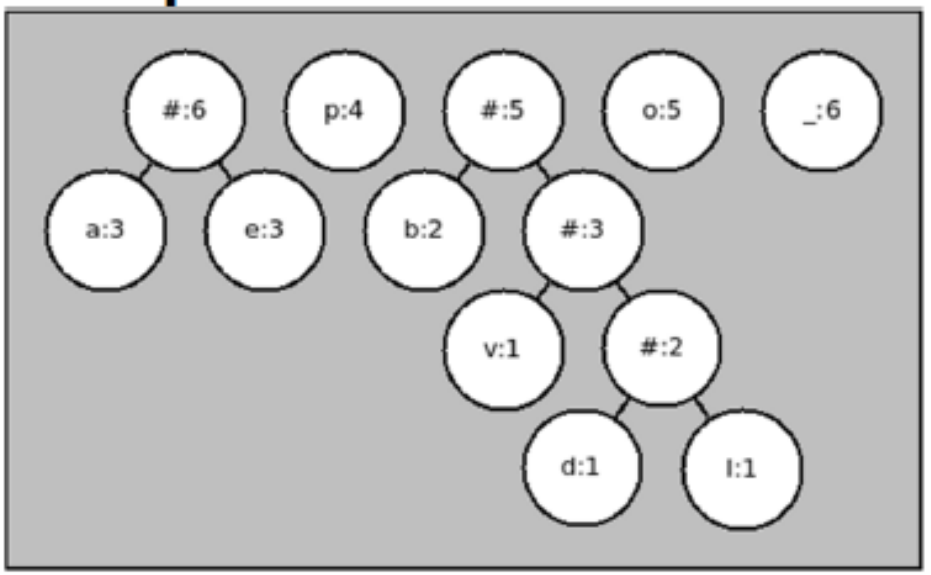



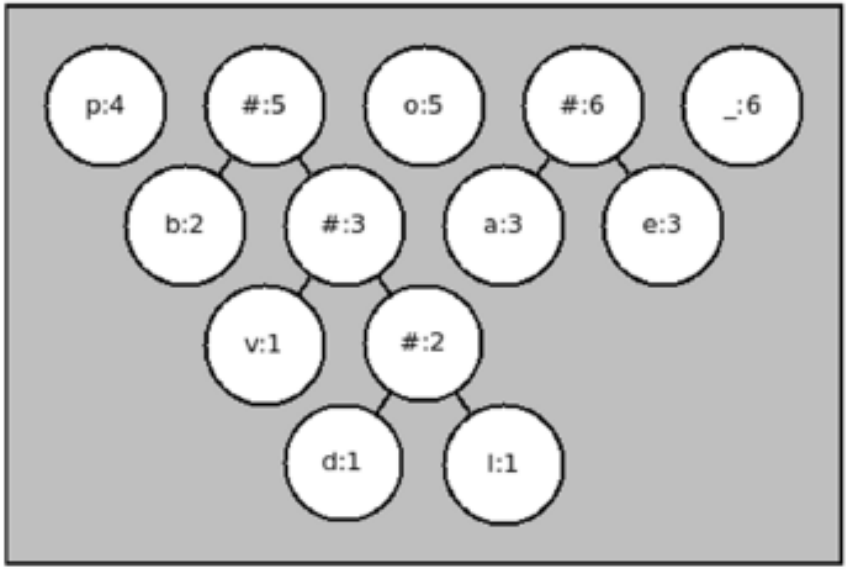
# Exemplos – Huffman



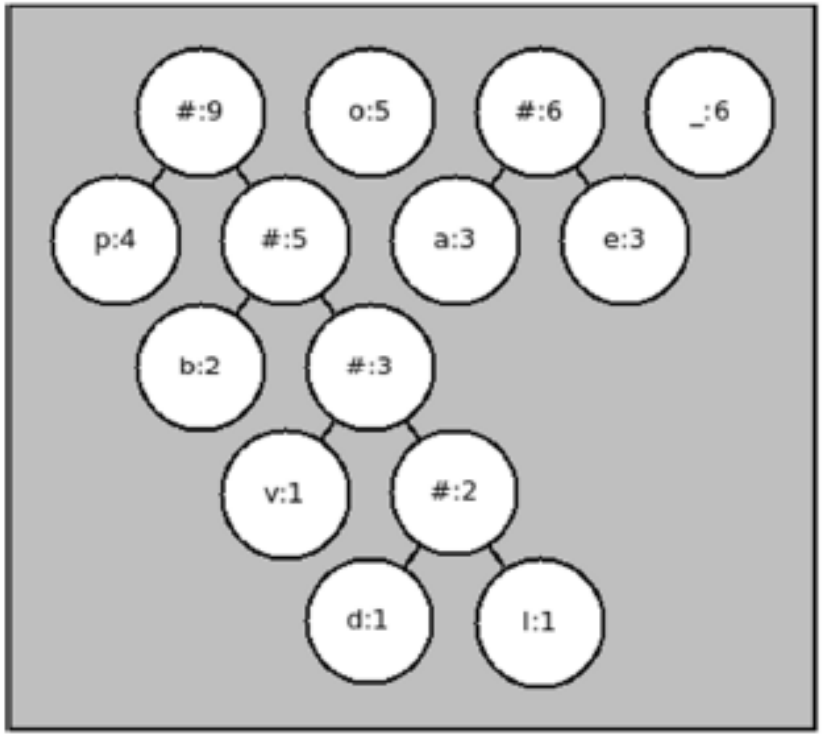



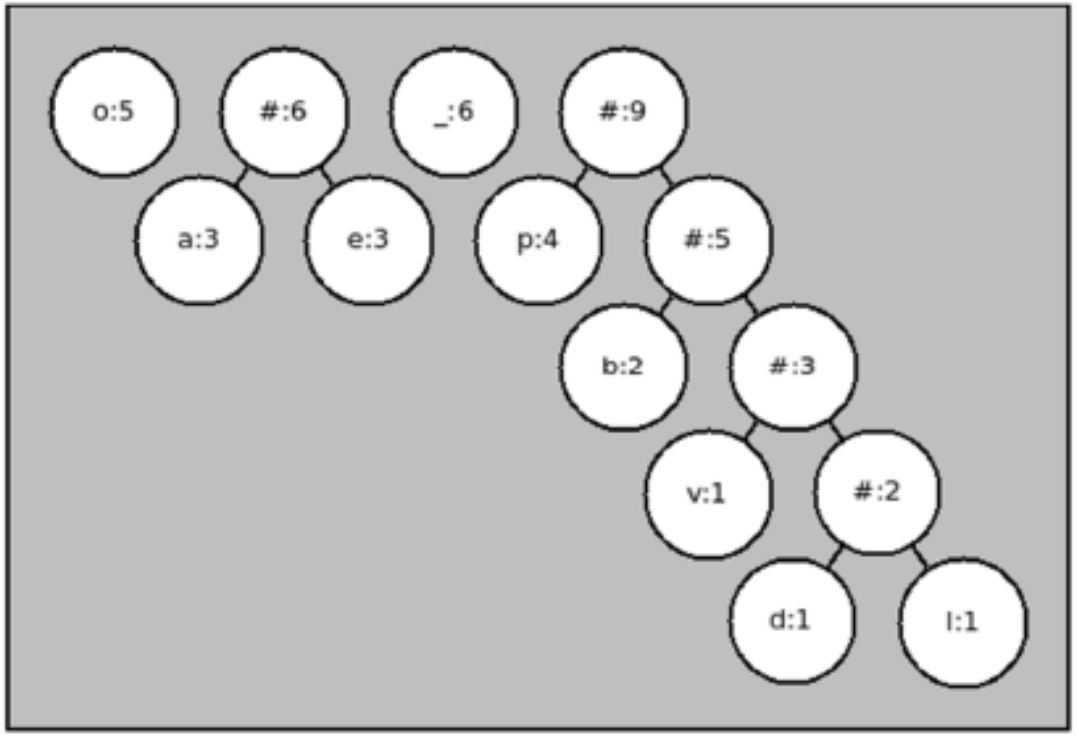
# Exemplos – Huffman



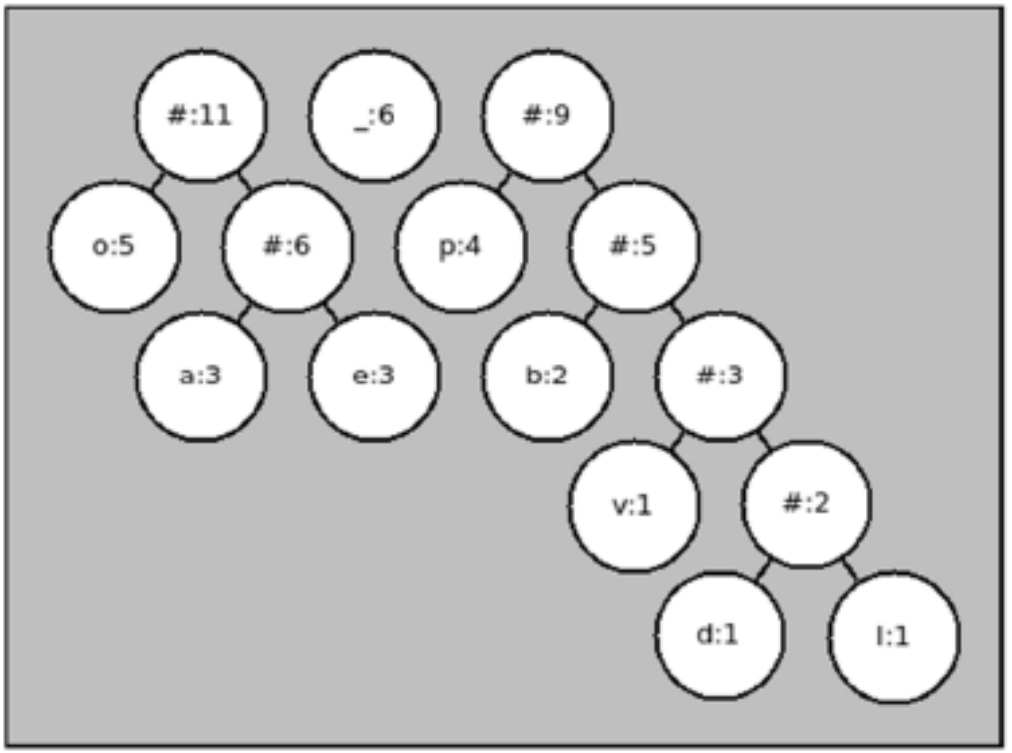



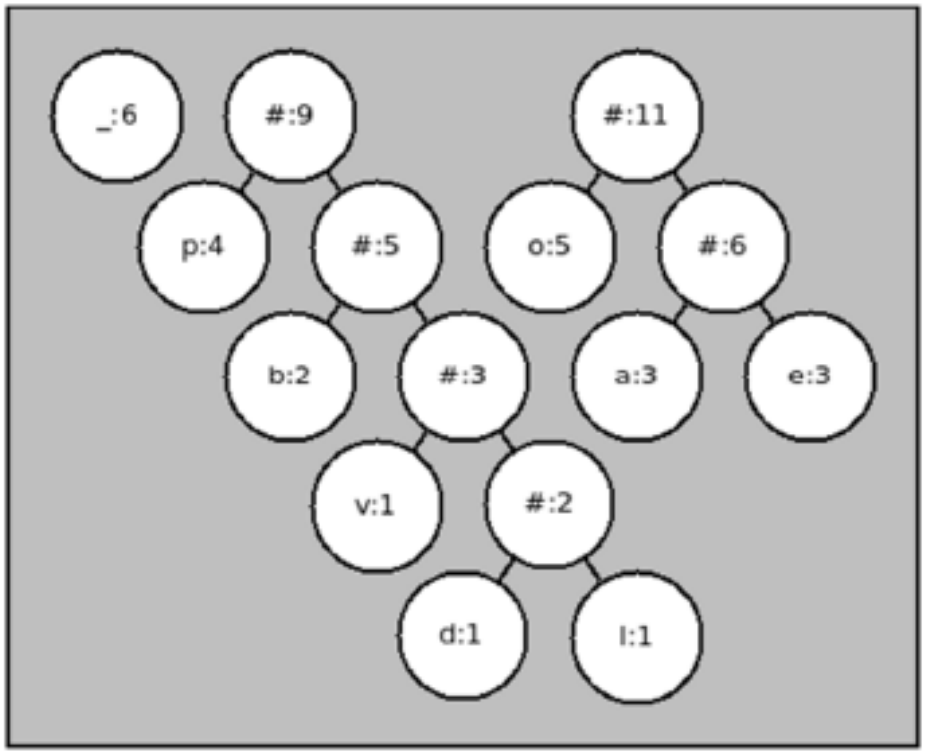
# Exemplos – Huffman



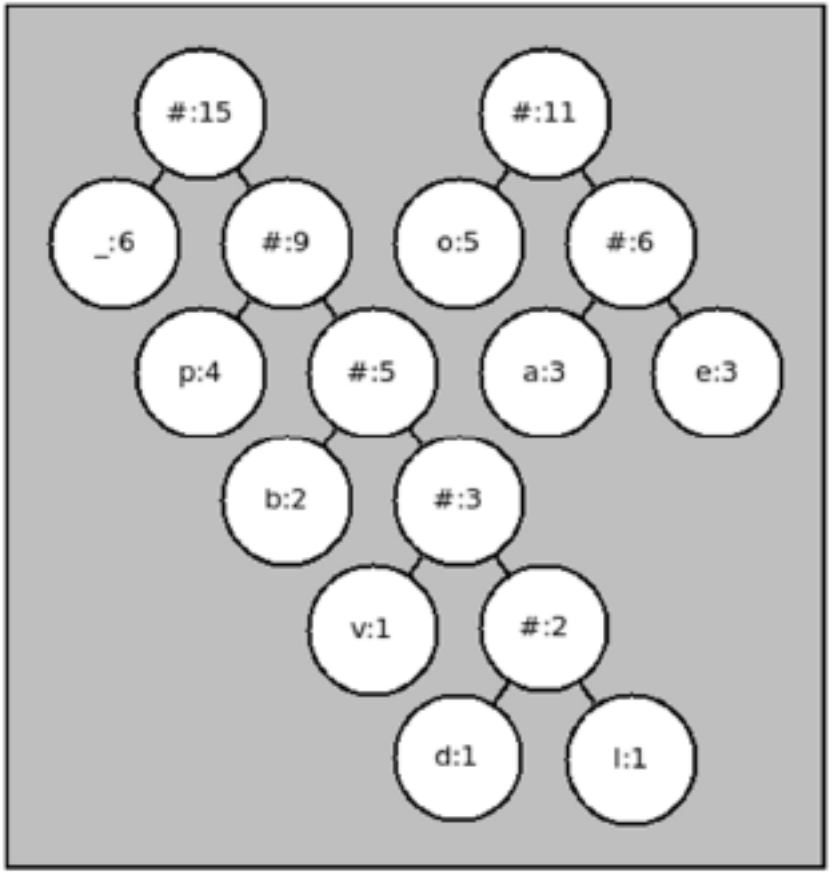



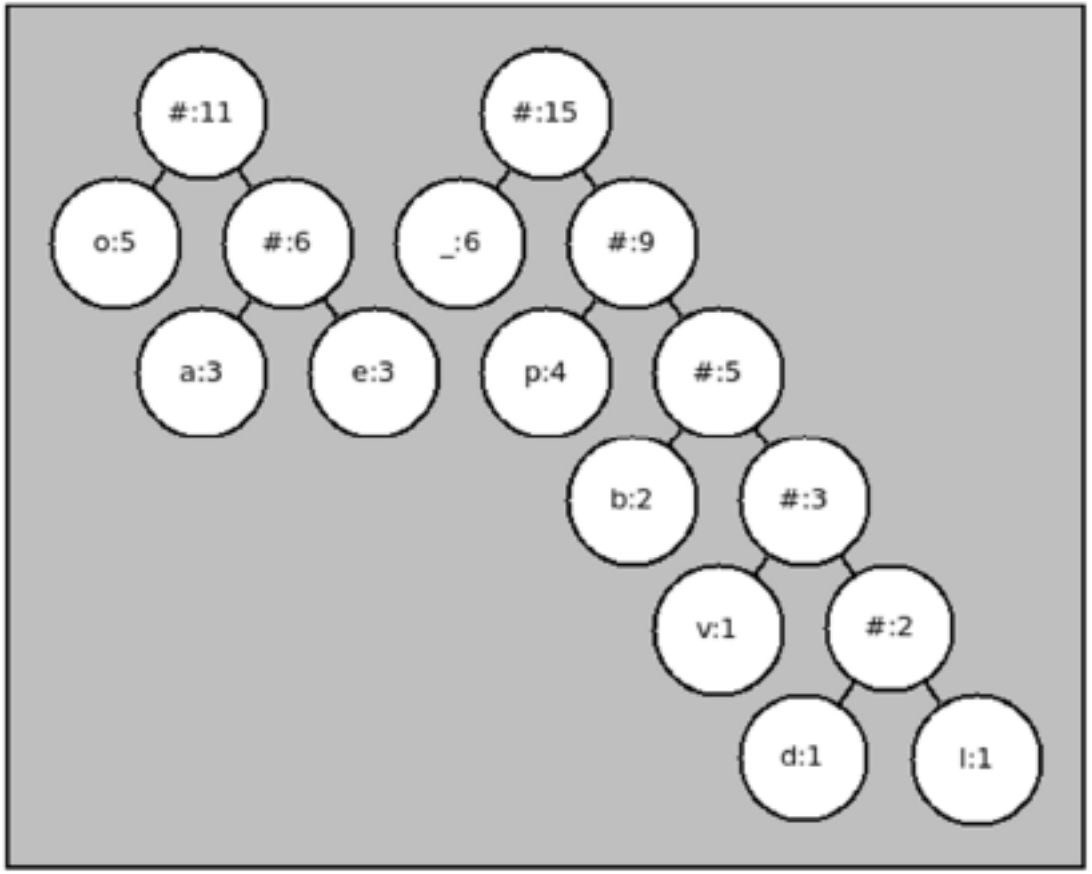
# Exemplos – Huffman



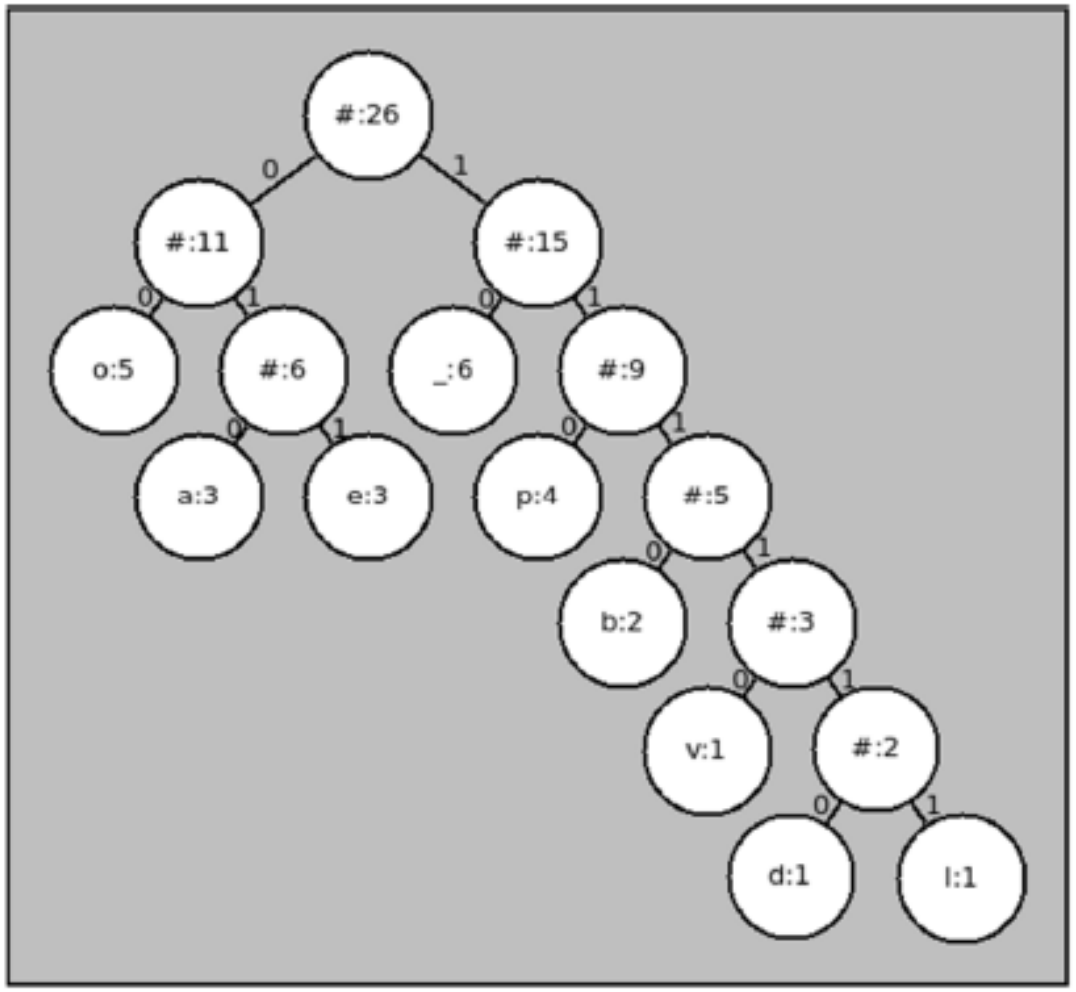



# Exemplos – Huffman



# Exemplos – Huffman




```
o:00

a:010

e:011

_:10

p:110

b:1110

v:11110

d:111110

l:111111
```

# Exemplos – Huffman

```
o:00
a:010
e:011
_:10
p:110
b:1110
v:11110
d:111110
l:111111
```

```
o - 5 ocorrências * 2 bits
_ - 6 ocorrências * 2 bits
p – 4 ocorrências * 3 bits
d – 1 ocorrência * 6 bits
a – 3 ocorrências * 3 bits
l – 1 ocorrência * 6 bits
e – 3 ocorrências * 3 bits
v – 1 corrência * 5 bits
b – 2 ocorrências * 4 bts

Total : 77 bits
```

# Exemplos – Huffman

Cabeçalho:

Numero de caracteres: 8 bits

9 caracteres * 8 bits = 72 bits

9 frequencias * 8 bits = 72 bits

Codificação:

77 bits

Total: 8 + 72 + 72 + 77 = 229 bits

Tamanho Original : 208 bits

Neste caso especifico, devido a ser pouco texto, não se consegue ter profeito com a codificação, mas em outras situações o algortimo consegue boas taxas de compatação

# Exemplos – Huffman

000010010111000100000101001000000000011001110000000
001000110000100000011011011000000000101100101000000
110111011000000001011000100000000100101100011000101
111100010110010110010110010101111110111111001010001
01110011111110011

Numero de Caracteres: 9

o5 _6 p4 a3 l1 e3 v1 b2

o popo do papa leva o bebe

```
o:00 a:010 e:011 _:10 p:110 b:1110 v:11110 d:111110 l:111111
```